How Angular 19+ (using the Vite/esbuild build system) improves performance when still trigger a full application reload for changes to TypeScript files ?

Piyali Das 3 min read · 6 hours ago



The key point is that **Angular 19+ isn't faster because of HMR for TypeScript — it's faster because the entire development build pipeline is much more efficient.**

Here's the breakdown.

Angular ≤16 (Webpack)

```
Save app.component.ts
        |
        ▼
Webpack recompiles dependency graph
        |
        ▼
Large bundle generation
        |
```

```
        ▼
Browser reload
        |
        ▼
Angular bootstrap
```

Webpack is bundle-centric. Even a small change often causes significant rebuild work.

Angular 17+ (esbuild + Vite)

```
Save app.component.ts
        |
        ▼
esbuild recompiles only affected modules
        |
        ▼
Vite serves updated module
        |
        ▼
Browser performs full reload
        |
        ▼
Angular bootstrap
```

Yes, there is still a **full page reload**, but notice what's missing:

- No expensive webpack bundling
- No rebuilding the whole dependency graph
- Much faster TypeScript transpilation
- Faster module serving

The reload itself usually takes only a few hundred milliseconds.

Where the performance comes from

1. esbuild is extremely fast

Webpack is written in JavaScript.

esbuild is written in Go.

For large projects:

```
| Builder | Rebuild Time |
| ------- | -----------: |
| Webpack |     6–15 sec |
| esbuild |    0.3–2 sec |
```

Angular now delegates TypeScript transpilation to esbuild.

2. Vite uses native ES Modules

Webpack:

```
src/
 ├── componentA
 ├── componentB
 ├── componentC

↓
bundle.js (5 MB)
```

Everything becomes one (or a few) bundles.

Vite:

```
Browser
   |
imports
   ▼
componentA.js
componentB.js
componentC.js
```

The browser loads only what it needs.

No huge bundle generation during development.

3. Incremental compilation

Suppose your project has:

```
500 components
150 services
80 directives
```

You edit

```
login.component.ts
```

Webpack may traverse much of the dependency graph.

esbuild recompiles only what changed.

4. Faster dev server startup

Typical startup times:

Angular 16

```
ng serve

Compiling...
20-40 seconds
```

Angular 20

```
ng serve

Ready in
2-6 seconds
```

5. Template HMR

Editing HTML:

```
Save
↓
Only template updated
↓
No page refresh
↓
Component state preserved
```

This wasn't possible before.

6. CSS HMR

Editing SCSS:

```
Save
↓
CSS injected
↓
No reload
↓
Animations continue
```

Why TypeScript still reloads

Imagine changing:

```
@Component({
  selector: 'app-login'
})
export class LoginComponent {
  constructor(private auth: AuthService) {}
  login() {}
}
```

Angular has to regenerate:

- Component metadata
- Dependency Injection graph
- Signals/effects (if used)
- Template instructions
- Component factory

Safely swapping that class while preserving state is difficult.

React can replace:

```
function App() {}
```

because components are plain JavaScript functions.

Angular components have compiled metadata and framework-managed lifecycle, making live replacement more complex.

Real-world comparison

Imagine a project with:

- 800 components
- 200 services
- 120 lazy modules

Angular 16

```
Change TS
↓
Webpack rebuild
8 s
↓
Reload
1 s
↓
Total ≈9 s
```

Angular 20

```
Change TS
↓
esbuild compile
0.8 s
↓
Reload
0.6 s
↓
Total ≈1.4 s
```

The application still reloads, but the wait is much shorter because the build step is dramatically faster.

Interview answer

If asked, “If Angular still reloads after TypeScript changes, why is Angular 19+ faster?”, you can answer:

> Angular 19+ improves development performance primarily by replacing the Webpack-based build pipeline with esbuild and Vite. Although TypeScript changes still trigger a full page reload, the rebuild is much faster because esbuild performs very fast incremental compilation and Vite serves native ES modules instead of rebuilding large bundles. In addition, template and stylesheet changes benefit from HMR, so many common UI edits no longer require any page reload at all.

This distinction is important: **HMR is only one aspect of the developer experience. The biggest performance gains in Angular 19+ come from the new build system itself.**

Angular + Programming + Technology + Coding + UI +

Written by Piyali Das

807 followers · 9 following

12 + yrs | Angular | NGRX | RxJS | https://www.linkedin.com/in/piyalidas10/
https://stackblitz.com/@piyalidas10 | https://github.com/piyalidas10

Edit profile